

ESTUDO DIRIGIDO: Gestão de Vulnerabilidades e Vazamento de Segredos (DevSecOps)

Ferramentas: Grafana, MergeStat, OSV-Scanner, Gitleaks e Grype

1. Objetivo

O objetivo deste estudo dirigido é demonstrar de forma prática como identificar, gerenciar e visualizar vulnerabilidades de segurança (CVEs) em códigos-fonte utilizando um conjunto de ferramentas *open-source* integrado. Este documento fornecerá um roteiro reproduzível passo a passo para que os alunos possam configurar um painel de monitoramento contínuo, facilitando a análise de riscos e a aplicação de mitigações em projetos de desenvolvimento de software.

2. Contextualização das Ferramentas

A segurança da cadeia de suprimentos de software tornou-se crítica. Para atacar esse problema adotando a abordagem "*shift-left*" (trazendo a segurança para as fases iniciais do desenvolvimento), utilizaremos o seguinte *stack* tecnológico:

- **OSV-Scanner:** Uma ferramenta de varredura de código aberto criada pelo Google. Ela examina as dependências declaradas no código-fonte em busca de vulnerabilidades (CVEs) conhecidas.
- **Grype:** Um scanner de vulnerabilidades especializado em imagens de contêineres e sistemas de arquivos, capaz de analisar pacotes de sistemas operacionais e bibliotecas.
- **Gitleaks:** Uma ferramenta de detecção projetada para encontrar segredos, como senhas, chaves de API e tokens esquecidos no repositório Git.
- **MergeStat:** Ferramenta responsável por sincronizar e coletar a saída desses scanners diretamente de repositórios Git, armazenando esses resultados estruturados em um banco de dados PostgreSQL.
- **Grafana:** Plataforma de visualização onde conectaremos o banco de dados PostgreSQL para criar painéis (dashboards) dinâmicos, permitindo o monitoramento unificado das falhas.

No contexto do pentest, este conjunto de ferramentas se aplica primordialmente à etapa de **Scanning (Varredura) e Enumeração**.

3. Missão de Pentest

- **Objetivo da análise:** Detectar e catalogar vulnerabilidades críticas (CVEs) em bibliotecas/contêineres e identificar vazamentos de dados sensíveis (segredos/tokens).
- **Tipo de alvo:** Repositório Git público, criado como laboratório experimental próprio, simulando o controle de versão corporativo de uma empresa que contém códigos legados, imagens de contêineres e possíveis credenciais esquecidas no código.
- **Hipótese de ataque ou investigação:** Um atacante pode obter acesso indevido à infraestrutura ao encontrar chaves de API expostas no código ou explorar

falhas conhecidas de execução remota presentes em componentes desatualizados.

- **Etapas do pentest relacionadas:** Scanning, Varredura de Vulnerabilidades e OSINT.
- **Resultado esperado:** Obtenção de um dashboard visual no Grafana que reporte as CVEs e os segredos vazados, fornecendo inteligência acionável.

4. Modelagem do Cenário

- **Estrutura do laboratório:** O ambiente de testes consiste em um repositório propositalmente vulnerável no GitHub, analisado por instâncias locais do Grafana e MergeStat executadas via Docker Compose.
- **Serviços envolvidos:** Uma instância do banco de dados PostgreSQL, o servidor do Grafana para acesso aos painéis, e uma instância do MergeStat rodando os syncs do OSV-Scanner, Grype e Gitleaks.
- **Riscos avaliados e Possíveis impactos:** Avalia-se o risco de comprometimento da aplicação devido ao vazamento de credenciais e uso de código de terceiros inseguro. O impacto pode variar desde vazamento de dados confidenciais até o comprometimento total do sistema.

5. Requisitos do Ambiente

Para reproduzir este laboratório, é necessário o seguinte ambiente:

- Docker e Docker Compose instalados na máquina local.
- Uma conta no GitHub para criar o repositório alvo.
- Acesso à internet para o download das imagens dos contêineres e sincronização.

6. Instalação e Configuração

A configuração divide-se na preparação do alvo e das ferramentas:

1. **Preparação do Alvo:** Crie um repositório público no GitHub. Adicione arquivos com vulnerabilidades conhecidas (ex: `package.json` antigo), um `Dockerfile` desatualizado e insira intencionalmente uma chave de API fictícia no código.
2. **Preparação das Ferramentas:** Utilize o Docker Compose para inicializar simultaneamente o banco de dados PostgreSQL, a interface do MergeStat e o servidor do Grafana em um ambiente isolado.

7. Execução Passo a Passo

Com os contêineres rodando, siga estes passos para iniciar a varredura:

1. Acesse a interface web do MergeStat local (ex: `http://localhost:3300`).
2. Navegue até a aba **Repos** e adicione a URL do seu repositório GitHub de teste.
3. Vá para a seção **Repo Syncs** e clique em **Add Sync**.
4. Configure os três scanners (OSV, Grype e Gitleaks) apontando para o repositório alvo.

5. Após os syncs finalizarem, acesse o Grafana (<http://localhost:3000>), adicione o PostgreSQL local como *Data Source* e importe o painel para consultar os resultados.

8. Comandos e Procedimentos Utilizados

Procedimentos exatos para a configuração:

- **Inicializar o ambiente:** `docker-compose up -d`
- **Ativar o OSV-Scanner:** No menu de "Add Sync", selecione o repositório e informe a imagem oficial: `ghcr.io/mergestat/sync-scan-osv:latest`.
- **Ativar o Gripe e Gitleaks:** Siga o mesmo procedimento de "Add Sync", apontando para as respectivas imagens de contêineres focadas em varredura de sistema de arquivos e histórico de commits.

9. Evidências da Execução

(Nota para os alunos: Durante a execução, os grupos deverão inserir aqui):

- [INSERIR: Print da tela do MergeStat mostrando os Syncs finalizados com sucesso]
- [INSERIR: Print da tela do Dashboard do Grafana exibindo o gráfico de pizza com a distribuição de vulnerabilidades por severidade]
- [INSERIR: Tabela ou log exportado mostrando uma chave de API detectada pelo Gitleaks]

10. Interpretação Técnica dos Resultados

Ao analisar o painel do Grafana, identificamos:

- **Riscos Encontrados:** O OSV-Scanner e o Gripe relataram CVEs de criticidade Alta e Crítica presentes nas dependências do projeto e na imagem base do Docker. O Gitleaks detectou com sucesso a chave de API inserida no histórico de commits do repositório alvo.
- **Relevância Técnica:** O painel oferece uma visão centralizada que ajuda as equipes a priorizar correções. Entender a exposição (ex: vulnerabilidades mais antigas ou atreladas a um ecossistema específico) permite focar nos repositórios críticos primeiro.
- **Limitações:** Scanners de análise estática podem gerar falsos positivos. Além disso, a análise foca apenas em componentes e código estático, não testando falhas dinâmicas de infraestrutura (como regras de firewall mal configuradas).

11. Mitigações e Contramedidas

Para solucionar as falhas detectadas:

- **Mitigação de CVEs:** Atualizar as bibliotecas e imagens de contêineres afetadas para versões mais recentes que possuam *patches* de segurança aplicados.

- **Mitigação de Segredos Vazados:** Revogar imediatamente a chave de API detectada pelo Gitleaks. Segredos não devem residir no código; recomenda-se a implementação de um cofre de senhas (como o HashiCorp Vault ou AWS Secrets Manager).
- **Estratégias de Defesa e Monitoramento:** Para evitar novos problemas, essas mesmas ferramentas (OSV, Gype e Gitleaks) devem ser acopladas diretamente na esteira de integração contínua (CI/CD). Isso permite bloquear (quebrar a *build*) qualquer solicitação de alteração no código (Pull Request) que introduza novas vulnerabilidades críticas ou tokens *hardcoded* antes que cheguem à produção.

12. Referências Técnicas

- **Tutorial de Integração (Grafana, MergeStat e OSV-Scanner):**
<https://grafana.com/blog/2023/05/26/how-to-manage-cve-security-vulnerabilities-with-grafana-mergestat-and-osv-scanner/>
- **Repositório Oficial do Gype:** <https://github.com/anchore/gype>
- **Repositório Oficial do Gitleaks:** <https://github.com/gitleaks/gitleaks>